Randomized DRL with Multi-Resource (CPU, RAM, Disk) Capacity: From Slots to Resource Vectors

October 2026

1 Problem Statement

1.1 Setting

We consider a Multi-access Edge Computing (MEC) network in which an edge node can cache services (code together with any associated data) downloaded from a remote cloud. A *service* s_n, $n = 1, \ldots, N$, is characterised by its demand vector

$$\mathbf{r}_n = (\mathrm{CPU}_n,\ \mathrm{RAM}_n,\ S_n) \in \mathbb{R}^R_{>0}, \qquad R = 3, \tag{1}$$

(CPU, RAM and disk footprint S_n), its round-trip forwarding latency l_n to the cloud, and its download time M_n (time to send the download request plus the time to download over the downlink). The edge node has capacity vector $\mathbf{c} = (C_{\mathrm{cpu}}, C_{\mathrm{ram}}, C_{\mathrm{disk}})$ and may cache at most C services. Throughout, resources are normalized:

$$a_{n,r} = \frac{r_{n,r}}{c_r} \in (0, 1], \qquad r \in \{\mathrm{cpu}, \mathrm{ram}, \mathrm{disk}\}, \tag{2}$$

so that every resource has capacity 1.

When a request for s_n arrives, exactly one of the following happens: (1) **Hit:** s_n is cached and served with zero latency; (2) **Request forwarding (R):** s_n is not cached and the request is forwarded to the cloud with latency l_n (the service may or may not be downloaded meanwhile); (3) **Delayed hit (D):** a download of s_n is in progress and the request is buffered if the remaining download time is below l_n, otherwise forwarded, so the latency is $\min(l_n, \text{remaining download time})$.

1.2 Deterministic DRL Problem and Online-DRL

Problem (DRL). *Design an online algorithm for caching and replacing services at the edge that makes no assumption on the request arrival pattern, accounts for delayed hits (D) and request forwarding (R), and minimises the total latency (L) while keeping the download cost below a given budget.*

The offline problem (OPT) with multi-resource capacity is

$$\min \quad \sum_n \sum_t d\big(s_n, r(s_n,t)\big) \tag{3}$$

$$\text{s.t.} \quad \sum_n a_{n,r}\, x_{n,t} \le 1, \quad r \in \{\text{cpu}, \text{ram}, \text{disk}\},\ \forall t, \tag{4}$$

$$\sum_n x_{n,t} \le C, \quad \forall t, \tag{5}$$

$$\sum_n \sum_t u_{n,t}\, M_n \le B_{\max}, \tag{6}$$

$$x_{n,t},\, y_{n,t},\, u_{n,t} \in \{0,1\}, \tag{7}$$

where $x_{n,t} = 1$ means s_n is cached, $y_{n,t} = 1$ that its download is in progress, $u_{n,t} = 1$ that a download is initiated at t (each download charged M_n once), and $B_{\max}$ is the download budget. This is a mixed integer program with a multi-dimensional knapsack structure per time step.

The deterministic algorithm **Online-DRL** downloads a service when the time since its first miss, or the latency accumulated by forwarded requests, reaches M_n, and evicts with a resource-aware Landlord policy. Its ratio $\gamma = \max_I \text{ALG}(I)/\text{OPT}(I)$ satisfies $\gamma \in O\big(k(1 + 3\beta_{\text{up}} M_x)\big)$ and $\gamma \in \Omega\big(k(1 + 3\beta_{\text{down}} M_{\min})\big)$, where k is the cache capacity, $M_x, M_{\min}$ are the largest and smallest download times and $\beta_{\text{up}}, \beta_{\text{down}}$ are the fractions of M_x and $M_{\min}$ for which requests are forwarded rather than buffered.

1.3 Motivation for Randomization

(i) Both bounds are linear in k; every deterministic paging algorithm has ratio at least k, while randomized algorithms achieve $O(\log k)$ against an oblivious adversary, so randomization may remove the linear dependence on k (a design goal, not a proved result here). (ii) The deterministic download threshold is a rent-or-buy rule that an adversary can exploit; randomizing the threshold improves the ski-rental ratio from 2 to $e/(e-1)$.

1.4 The Randomized DRL Problem

Problem (Randomized DRL, multi-resource). *Design a randomized online algorithm for caching and replacing services at the edge that (i) makes no assumption on the request arrival pattern beyond an oblivious adversary, (ii) accounts for delayed hits (D) and request forwarding (R), (iii) minimises the expected total latency, and (iv) keeps the expected download cost below $B_{\max}$ while respecting the CPU, RAM and disk capacities directly, without first collapsing them into a single slot count.*

The algorithm has three randomized components.

(R1) Fractional state. Marginal caching probabilities $P_t \in \mathcal{D}_{\text{vec}}$ and download probabilities Q_t, with P_t updated by projected online gradient descent on a latency surrogate (Section 3), where

$$\mathcal{D}_{\text{vec}} = \Big\{ P \in [0,1]^N \ :\ \sum_n a_{n,r} p_n \le 1 \ \ \forall r, \ \ \sum_n p_n \le C \Big\}. \tag{8}$$

(R2) Randomized rounding. K sample paths $s_{k,\cdot,t} \in \{0,1\}^N$ with $\mathbb{E}[s_{k,n,t}] = p_{n,t}$, followed by a multi-resource feasibility repair (Section 2.5); the serving path is $k^* \sim \text{Uniform}\{1,\dots,K\}$.

(R3) Randomized download trigger. A threshold $\theta_n \in [0, M_n]$ with density $f(\theta) = e^{\theta/M_n}/(M_n(e-1))$ replaces the deterministic threshold M_n.

Formal goals (expectation over internal randomness, oblivious adversary):

(G1) Feasibility:
$$\sum_n a_{n,r} x_{n,t} \le 1 \ \forall r, \quad \sum_n x_{n,t} \le C \quad \text{(after repair, almost surely);} \tag{9}$$

(G2) Budget:
$$\mathbb{E}\Big[\sum_{n,t} u_{n,t}\, M_n\Big] \le B_{\max}; \tag{10}$$

(G3) Regret:
$$\mathbb{E}\Big[\sum_t d_t(n_t^*)\Big] - \min_{P \in \mathcal{D}_{\text{vec}}} \sum_t \hat{F}_t(P) \le \text{Reg}_T = o(T); \tag{11}$$

(G4) Expected ratio:
$$\gamma_{\text{R}} = \max_I \ \mathbb{E}[\text{ALG}_{\text{R}}(I)]/\text{OPT}(I) \text{ below the deterministic } \gamma; \tag{12}$$

(G5) Conversion loss: if a slot reduction is used, its capacity loss Λ_ϕ (Section 2.4) is reported. (13)

2 Converting CPU, RAM and Disk into Slots: Mathematical Tools

The earlier formulation gave each service a scalar slot weight w_n and a single constraint $\sum_n w_n p_n \le C_{\text{slot}}$. A slot reduction is a map

$$w_n = \phi(a_{n,\text{cpu}}, a_{n,\text{ram}}, a_{n,\text{disk}}) \cdot C_{\text{slot}}. \tag{14}$$

It is *safe* (an inner approximation) if $\sum_n w_n p_n \le C_{\text{slot}}$ implies $\sum_n a_{n,r} p_n \le 1$ for every r, which holds whenever $\phi(\mathbf{a}_n) \ge \max_r a_{n,r}$. Below are the tools available, from simplest to most exact.

2.1 Dominant-Resource (Max-Norm) Slots

$$w_n^{\infty} = C_{\text{slot}} \max_r a_{n,r}. \tag{15}$$

A service occupies as many slots as its most-demanding resource, as in Dominant Resource Fairness. It is always safe, since $\sum_n a_{n,r} p_n \le \sum_n \max_{r'} a_{n,r'} p_n \le 1$. Its drawback is waste: a CPU-heavy service reserves disk and RAM capacity it does not use.

2.2 Weighted Linear Scalarization

$$w_n^{\text{lin}} = C_{\text{slot}} \sum_r \alpha_r a_{n,r}, \qquad \alpha \in \Delta^{R-1} \text{ (or } \alpha_r \ge 1 \text{ for safety).} \tag{16}$$

With α on the simplex this is not safe: a service using only disk can look cheap. It is safe if $\alpha_r \geq 1$ for all r (then $w_n^{\text{lin}} \geq w_n^{\infty}$), at the cost of up to a factor R of wasted capacity. The weights α can be chosen by:

- *AHP (analytic hierarchy process):* pairwise-comparison matrix whose principal eigenvector gives α;
- *Entropy weighting:* $\alpha_r \propto 1 - H_r$, where H_r is the normalized entropy of the demands $\{a_{n,r}\}_n$, so resources with more discriminating demands get larger weight;
- *Utilization-based weights:* $\alpha_r \propto$ observed scarcity (average utilization of resource r).

2.3 ℓ_p-Norm Interpolation

$$w_n^{(p)} = C_{\text{slot}} \Big(\sum_r \alpha_r a_{n,r}^p \Big)^{1/p}, \qquad p \in [1, \infty]. \tag{17}$$

This family contains the previous two: $p = 1$ is linear scalarization and $p \to \infty$ with $\alpha_r = 1$ is the dominant-resource rule. With $\alpha_r = 1$ it satisfies $w_n^{\infty} \leq w_n^{(p)} \leq R^{1/p} w_n^{\infty}$, so it is safe, and p trades waste (p small) against tightness (p large).

2.4 Quantifying the Loss of the Slot Reduction

For a reduction ϕ define its capacity loss

$$\Lambda_\phi = \max_n \frac{\phi(\mathbf{a}_n)}{\max_r a_{n,r}} \quad \geq 1, \tag{18}$$

so a safe reduction can waste at most a factor Λ_ϕ of capacity ($\Lambda = 1$ for dominant-resource slots, $\Lambda \leq R^{1/p}$ for ℓ_p with $\alpha_r = 1$). If slots must be integers, $w_n = \lceil C_{\text{slot}} \phi(\mathbf{a}_n) \rceil$ adds a rounding loss of at most one slot per service. The slot formulation is the right choice only when Λ_ϕ is close to 1 (balanced demands); otherwise the vector formulation below is preferable.

2.5 Keeping the Resources Separate: Lagrangian Pricing and Vector Bin Packing

- **Shadow prices.** Relax the three capacity constraints with multipliers $\boldsymbol{\rho} = (\rho_{\text{cpu}}, \rho_{\text{ram}}, \rho_{\text{disk}}) \geq 0$. The *effective slot weight* of a service is its priced cost $w_n(\boldsymbol{\rho}) = \sum_r \rho_r a_{n,r}$, learned from the LP dual instead of fixed in advance. Scarce resources get high prices, so the reduction adapts to the workload. This is the exact approach used in Section 3 (multi-multiplier projection).
- **Vector bin packing heuristics.** For the repair step: order candidates by the size measure $\|\mathbf{a}_n\|_2$ or by alignment $\langle \mathbf{a}_n, \mathbf{u} \rangle$ with the residual-capacity vector $\mathbf{u}$ (Tetris-style), and add or evict greedily. Packing in R dimensions has no better than $O(\log R)$-type approximation guarantees in general, which is why a repair step is used instead of exact feasibility.

- **Resource augmentation.** Allow the realized state to use $(1+\epsilon)$ of each capacity, which makes randomized rounding feasible with high probability (Chernoff bounds on $\sum_n a_{n,r} x_{n,t}$ when the rounding is independent across services).

Tool	Weight / constraint	Safe?	Waste factor
Dominant resource	$\max_r a_{n,r}$	Yes	$\Lambda = 1$; other resources may sit idle
Linear scalarization	$\sum_r \alpha_r a_{n,r}$	Only if $\alpha_r \geq 1$	up to R
ℓ_p norm	$(\sum_r \alpha_r a_{n,r}^p)^{1/p}$	Yes for $\alpha_r = 1$	$\leq R^{1/p}$
Shadow prices	$\sum_r \rho_r a_{n,r}$	Yes at optimal $\boldsymbol{\rho}$	none (exact)
Vector bin packing	separate constraints	Yes (repair)	heuristic

Table 1: Slot-conversion tools.

3 Probability Update with Vector Resource Constraints

3.1 Instantaneous Gradient ($\nabla_{P_t} \hat{F}_t$)

For a request for n^* at time t (one-hot arrival vector),

$$\nabla_{p_{n,t}} = \begin{cases} -\Big[(1 - q_{n^*,t})\, l_{n^*} + q_{n^*,t} \min\big(l_{n^*}, \Delta m_{n^*,t}\big)\Big], & n = n^*, \\ 0, & n \neq n^*, \end{cases} \tag{19}$$

which is $-l_{n^*}$ when $q_{n^*,t} = 0$. This is unchanged by the resource model.

3.2 Intermediate Update (Z_t)

$$z_{n,t} = p_{n,t-1} - \eta \nabla_{p_{n,t}}. \tag{20}$$

3.3 Projection onto the Multi-Resource Polytope

Write the constraints of $\mathcal{D}_{\text{vec}}$ as $\mathbf{A}P \leq \mathbf{b}$ with rows $r = 0, 1, 2, 3$: $a_{n,0} = 1$, $b_0 = C$ (cardinality) and $a_{n,r}$, $b_r = 1$ for CPU, RAM, disk. The projection is

$$P_t = \Pi_{\mathcal{D}_{\text{vec}}}(Z_t) = \arg \min_{P \in [0,1]^N,\ \mathbf{A}P \leq \mathbf{b}} \ \tfrac{1}{2} \|P - Z_t\|_2^2. \tag{21}$$

KKT solution. With multipliers $\boldsymbol{\rho} = (\rho_0, \rho_{\text{cpu}}, \rho_{\text{ram}}, \rho_{\text{disk}}) \geq 0$,

$$p_{n,t}(\boldsymbol{\rho}) = \min\Big(1,\ \max\Big(0,\ z_{n,t} - \sum_{r=0}^{3} \rho_r\, a_{n,r}\Big)\Big), \tag{22}$$

with complementary slackness $\rho_r\big(\sum_n a_{n,r} p_{n,t} - b_r\big) = 0$. Compared with the slot case, the single threshold $\rho\, w_n$ is replaced by the priced cost $\sum_r \rho_r a_{n,r}$; setting all multipliers but one to zero recovers the slot formula with $w_n = a_{n,r}$.

Computing $\boldsymbol{\rho}$. The dual function

$$g(\boldsymbol{\rho}) = \min_{P\in[0,1]^N} \ \tfrac{1}{2}\|P - Z_t\|^2 + \sum_r \rho_r \Big(\sum_n a_{n,r} p_n - b_r \Big) \tag{23}$$

is concave and differentiable with $\nabla_{\rho_r} g = \sum_n a_{n,r} p_{n,t}(\boldsymbol{\rho}) - b_r$. Slater's condition holds ($P = 0$ is strictly feasible), so there is no duality gap, and $\boldsymbol{\rho}^\star$ is found by projected dual ascent

$$\rho_r \leftarrow \Big[\rho_r + \kappa \Big(\sum_n a_{n,r} p_{n,t}(\boldsymbol{\rho}) - b_r \Big) \Big]_+, \tag{24}$$

at $O(N \cdot 4)$ per iteration; alternatives are ADMM or Dykstra's alternating projections. When only one constraint is active, the exact $O(N \log N)$ breakpoint sort of the slot case applies. After stopping at tolerance ϵ, a final scaling

$$P_t \leftarrow P_t \Big/ \max\Big(1, \max_r \tfrac{\sum_n a_{n,r} p_{n,t}}{b_r}\Big) \tag{25}$$

keeps $P_t \in [0,1]^N$ and makes it exactly feasible, because all constraints are homogeneous and linear.

3.4 Rounding with Repair (R2)

Draw sample paths with marginals $p_{n,t}$ (consecutive paths coupled so the expected number of changes is $O(\|P_t - P_{t-1}\|_1)$). If a sampled state violates some $\sum_n a_{n,r} x_{n,t} \le 1$, evict sampled services in increasing order of $p_{n,t} / \max_r a_{n,r}$ until all three resources are feasible. Without repair, feasibility holds only in expectation: $\mathbb{E}[\sum_n a_{n,r} x_{n,t}] = \sum_n a_{n,r} p_{n,t} \le 1$.

3.5 Regret

Assumption 1. *$q_{n,t}$ and $\Delta m_{n,t}$ are treated as exogenous in $\hat{F}_t$, and cache and download indicators are conditionally independent given (P_t, Q_t).*

Proposition 1. *Under Assumption 1, $\hat{F}_t$ is linear, $\|\nabla \hat{F}_t\|_2 \le l_{\max}$ and the diameter of $\mathcal{D}_{\text{vec}}$ is at most $\sqrt{N}$. Projected online gradient descent with $\eta = \sqrt{N}/(l_{\max}\sqrt{T})$ gives regret at most $\sqrt{N}\, l_{\max}\sqrt{T}$ against the best fixed $P \in \mathcal{D}_{\text{vec}}$. If a safe slot reduction is used instead, the comparator set shrinks to $\mathcal{D}_{\text{slot}} \subseteq \mathcal{D}_{\text{vec}}$, and the regret is measured against a weaker benchmark.*

4 Latency Model and Updated Metrics

4.1 Downloading Time

$$M_n = M_n^{\text{req}} + \frac{S_n}{D}, \qquad m_n = \Big\lceil \frac{M_n}{\tau} \Big\rceil, \tag{26}$$

where $S_n = r_{n,\text{disk}}$ is the disk component of $\mathbf{r}_n$, D the downlink bandwidth and τ the slot duration.

4.2 Physical Latency

$$d_t(n^*) = \begin{cases} 0, & x_{n^*,t} = 1, \\ \min\left(l_{n^*}, \Delta m_{n^*,t}\right), & x_{n^*,t} = 0,\ y_{n^*,t} = 1, \\ l_{n^*}, & x_{n^*,t} = 0,\ y_{n^*,t} = 0, \end{cases} \qquad \Delta m_{n^*,t} = \max\{0, t_{\text{start}} + m_{n^*} - t\}. \tag{27}$$

4.3 Expected Latency and Surrogate

$$\mathbb{E}[d_t(n^*)] = (1 - p_{n^*,t})\Big[(1 - q_{n^*,t})l_{n^*} + q_{n^*,t}\min\left(l_{n^*}, \Delta m_{n^*,t}\right)\Big], \tag{28}$$

$$\hat{F}_t(P) = \sum_n \lambda_{n,t}(1 - p_n)\Big[(1 - q_{n,t})l_n + q_{n,t}\min(l_n, \Delta m_{n,t})\Big]. \tag{29}$$

4.4 Resource Metrics Replacing the Slot Count

Instead of "slots used", the following quantities are tracked for the realized state X_t.

$$\text{Utilization of resource } r: \quad U_{r,t} = \sum_n a_{n,r}x_{n,t} \in [0,1], \tag{30}$$

$$\text{Bottleneck utilization:} \quad U_t^{\max} = \max_r U_{r,t}, \tag{31}$$

$$\text{Imbalance:} \quad I_t = \max_r U_{r,t} - \min_r U_{r,t}, \tag{32}$$

$$\text{Stranded capacity:} \quad W_t = \sum_r \left(U_t^{\max} - U_{r,t}\right), \tag{33}$$

$$\text{Expected latency per unit resource:} \quad \frac{\mathbb{E}[\sum_t d_t]}{\sum_t U_t^{\max}}. \tag{34}$$

$U_t^{\max}$ is exactly the dominant-resource slot occupancy divided by C_{slot}; the imbalance and stranded capacity W_t show how much the slot reduction would have wasted, so they give an empirical estimate of Λ_ϕ.